

ADVANCED JAVA PROGRAMMING COURSE

Basic Java

From First Program to Control Flow & Loops

Faculty of Information Technologies
Industrial University of Ho Chi Minh City

Chapter Contents

01 Introduction — Java, JVM, JDK, first program

02 Variables, Types & Operators

03 Input / Output & String Basics

04 Control Structures — if, else, switch

05 Loops — while, do-while, for, break, continue

06 Review, Combined Exercises & Next Steps

01 · INTRODUCTION

Java & Development Environment

Language · Platform · Tools · First program

What is Java?

Programming language

Object-oriented, strongly typed, general-purpose

Platform

Runs on the Java Virtual Machine (JVM)

Write once, run anywhere

Same bytecode on Windows, macOS, Linux...

Widely used

Backend, Android, enterprise systems, tools

JDK · JRE · JVM

JVM

Java Virtual Machine — executes bytecode (.class)

JRE

Runtime Environment — JVM + libraries to run apps

JDK

Development Kit — JRE + javac compiler + tools

How a Java Program Runs



1

Source

Hello.java



2

Compile

javac → .class



3

Bytecode

Platform-neutral



4

Run

java Hello

Tools You Need

JDK 17 or 21

Oracle JDK or Eclipse Temurin (Adoptium)

IDE

IntelliJ IDEA Community · VS Code · Eclipse

Check install

`java -version` · `javac -version`

Optional

Terminal / command line for `javac` and `java`

First Program — HelloWorld

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

// File name must be HelloWorld.java

// javac HelloWorld.java → java HelloWorld

Structure of a Simple Program

class

At least one class per program

main

Entry point: `public static void main(String[] args)`

File name

Must match the public class name + `.java`

Statements

End with semicolon `;`

Blocks

Code inside `{ }`

Comments

`//` one line `/*` multi-line `*/`

Common Beginner Mistakes

- 01 File name $\neq$ public class name $\rightarrow$ compile error
- 02 Forgot semicolon at end of statement
- 03 Wrong main signature (missing static or `String[] args`)
- 04 Java is case-sensitive: `System` $\neq$ `system`
- 05 Run class name only — not `HelloWorld.java`

02 · VARIABLES & OPERATORS

Variables, Types & Operators

Data · Memory · Calculations · Comparisons

Variables

A named place in memory that holds a value

```
int age = 20;
```

```
String name = "An";
```

```
double score = 8.5;
```

```
int count; // declare
```

```
count = 10; // assign later
```

```
final int MAX = 100; // constant
```

Primitive Types

byte / short / int / long

Integers — int is the default choice

float / double

Decimals — double is the default

char

Single character: 'A', '1'

boolean

true or false

Type Casting

Widening (safe)

Smaller → larger
Automatic.

```
int i = 10;  
double d = i; // OK
```

Narrowing (explicit)

Larger → smaller
Must cast — may lose data.

```
double d = 9.7;  
int i = (int) d; // 9
```

Operators Overview

Arithmetic

+ - * / % · ++ --

Comparison

== != > < >= <= → boolean

Logical

&& || ! (short-circuit)

Assignment

= += -= *= /= %=

Watch Out — Integer Division

```
int a = 5, b = 2;
```

```
System.out.println(a / b);    // 2 (not 2.5)
```

```
System.out.println((double) a / b); // 2.5
```

```
System.out.println(a / 2.0);    // 2.5
```

```
System.out.println(a % b);    // 1 (remainder)
```

03 · INPUT / OUTPUT & STRING

Input, Output & String Basics

Scanner · print · String methods

Output — System.out

```
System.out.print("Hi");    // no new line
```

```
System.out.println("Hi"); // with new line
```

```
System.out.printf("Age: %d%n", 20);
```

```
// Common format specifiers
```

```
// %d integer  %f decimal  %s string  %n new line
```

Input — Scanner

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Name: ");
String name = sc.nextLine();
System.out.print("Age: ");
int age = sc.nextInt();
sc.close();
```

Scanner Methods

`next()`

Next word (stops at space)

`nextLine()`

Whole line including spaces

`nextInt()`

Read an int

`nextDouble()`

Read a double

`nextBoolean()`

Read true / false

String Basics

```
String s = "Hello";  
  
s.length()      // 5  
  
s.charAt(0)      // 'H'  
  
s.equals("Hello") // true (use equals, not ==)  
  
s.toUpperCase()  // "HELLO"  
  
s + " World"     // "Hello World"
```

Common Scanner Pitfall

```
int age = sc.nextInt();
```

```
String name = sc.nextLine(); // may read empty!
```

```
// nextInt leaves the newline in the buffer.
```

```
// Fix: consume it first
```

```
int age = sc.nextInt();
```

```
sc.nextLine(); // clear leftover newline
```

```
String name = sc.nextLine();
```

04 · CONTROL STRUCTURES

if · else · switch

Making decisions in code

if · if-else · else if

```
if (score >= 5) {  
    System.out.println("Pass");  
} else if (score >= 0) {  
    System.out.println("Fail");  
} else {  
    System.out.println("Invalid");  
}
```

Ternary Operator ? :

// Short form of simple if-else

```
String result = (score >= 5) ? "Pass" : "Fail";
```

// Same as:

```
String result;
```

```
if (score >= 5) result = "Pass";
```

```
else result = "Fail";
```

switch

```
switch (day) {  
    case 1: System.out.println("Mon"); break;  
    case 2: System.out.println("Tue"); break;  
    case 3: System.out.println("Wed"); break;  
    default: System.out.println("Other");  
}  
  
// Always put break (unless fall-through is intended)
```

Comparing Values Correctly

Numbers / boolean

Use `==` `!=` `>` `<`

String content

Use `equals()` — not `==`

String ignore case

`equalsIgnoreCase()`

null-safe tip

`"text".equals(s)` preferred

05 · LOOPS

while · do-while · for

Repeat actions · break · continue

while

Condition checked first — may run zero times

```
int i = 1;  
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}  
// Prints 1 2 3 4 5
```

do-while

Body runs at least once — condition checked after

```
int n;  
  
do {  
    System.out.print("Enter positive: ");  
    n = sc.nextInt();  
} while (n <= 0);
```

for

Best when the number of iterations is known

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

// init ; condition ; update

// i is scoped inside the loop

break & continue

break

Exit the loop immediately.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    // stops at 5  
}
```

continue

Skip this iteration,
continue with next.

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    // skips printing 3  
}
```

Nested Loops — Example

```
// Multiplication table 1..5  
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= 5; j++) {  
        System.out.print(i * j + "\t");  
    }  
    System.out.println();  
}
```

Which Loop to Choose?

Known count (1 to n)

for

Unknown count, check first

while

Must run at least once

do-while

Stop early on condition

break inside any loop

Skip one iteration

continue

06 · REVIEW & NEXT STEPS

Review · Exercises · What's Next

Putting it all together

What We Covered

1. Java platform: JDK, JRE, JVM, first program
2. Variables, primitive types, casting, operators
3. Scanner input, System.out, String basics
4. if / else / switch and correct comparisons
5. while, do-while, for, break, continue

Practice Ideas

01 Grade calculator — input scores, print Pass/Fail

02 Menu with switch — 1.Add 2.Sub 3.Exit

03 Guess the number — loop until correct

04 Print multiplication table (nested for)

05 Sum of digits of an integer (while)

06 Check prime number (for + if)

What's Next?

Methods

Write reusable blocks of code

Arrays

Store many values of the same type

OOP basics

class, object, fields, methods

ArrayList

Dynamic lists of objects

Thank you

Questions?

Faculty of Information Technologies · IUH